

Job Referral Network (CognoDB Graph Database Application)

A modern full-stack web application demonstrating the power of graph databases by building a professional job referral network. Users can discover job opportunities not just by their skills, but by leveraging their 1st, 2nd, and 3rd degree connections.

Why Graph Database?

Relational databases struggle with highly interconnected data. In a standard SQL database, answering the question: *"Which jobs are my friends' friends' companies hiring for, that match my skills?"* requires multiple expensive, slow JOIN operations across several tables (Users, Connections, Companies, Jobs, Skills, etc.).

Graph Databases (CognoDB / Neo4j) excel at this because:

1. **Relationships are first-class citizens:** Connections between data points are stored natively, making traversals instantaneous.
2. **Multi-hop querying is trivial:** A 3-hop query (User -> Friend -> Company -> Job) is expressed simply and executes in milliseconds, regardless of total dataset size.
3. **Agile Schema:** As our network grows (e.g., adding Universities or Certifications), we can easily add new node and relationship types without complex migrations.

Graph Data Model

Nodes:

- User: (id, name, email, experience)
- Company: (id, name, location)
- Job: (id, title, location, salary)
- Skill: (id, name)

Relationships:

- (User) -[:KNOWS]-> (User)
- (User) -[:WORKS_AT]-> (Company)
- (User) -[:HAS_SKILL]-> (Skill)
- (Company) -[:HIRING_FOR]-> (Job)
- (Job) -[:REQUIRES]-> (Skill)

Architecture

- **Frontend:** React, Vite, Tailwind CSS, React Router, Axios
- **Backend:** Node.js, Express.js, Neo4j Official JavaScript Driver (neo4j-driver)
- **Database:** CognoDB (managed openCypher graph database)
- **Design Pattern:** MVC architecture on the backend (Routes -> Controllers -> Services)

Folder Structure

```
├── backend/
│   ├── config/           # DB connection
│   ├── controllers/      # Route logic
│   ├── middlewares/      # Error handling
│   ├── routes/           # Express routers
│   ├── seed/             # Data generation script
│   ├── services/         # Cypher queries
│   ├── app.js            # Express app setup
│   └── server.js         # Entry point
├── frontend/
│   ├── src/
│   │   ├── pages/        # React views
│   │   ├── services/     # Axios API
│   │   ├── App.jsx       # Routing
│   │   └── main.jsx
│   ├── tailwind.config.js
│   └── package.json
└── README.md
```

Setup & Installation

Prerequisites

- Node.js (v16+)
- A free [CognoDB Cloud \(https://console.cognoDB.com/signup\)](https://console.cognoDB.com/signup) instance.

1. Environment Variables

In the backend/ directory, create a .env file based on .env.example:

```
DB_URI=bolt+s://<your-instance-id>.databases.cognoDB.cloud
DB_USERNAME=cognoDB
DB_PASSWORD=your_password
PORT=5000
```

2. Backend Setup

```
cd backend
npm install
# Seed the database with mock data
node seed/seed.js
# Start the development server
npm run dev
```

3. Frontend Setup

```
cd frontend
npm install
npm run dev
```

The app will be available at <http://localhost:5173>.

Main Cypher Queries Explained

Here is an example of a powerful graph query used in the **Recommendations** feature:

```
MATCH (u:User {id: $userId})-[:HAS_SKILL]->(s:Skill)<-[:REQUIRES]-(j:Job)<-[:HIRING_FOR]-(c:Company)
OPTIONAL MATCH (u)-[:KNOWS]-(connection:User)-[:WORKS_AT]->(c)
RETURN j AS job, c AS company, collect(s) AS matchedSkills, collect(DISTINCT connection) AS contacts
ORDER BY size(matchedSkills) DESC, size(contacts) DESC
```

What this does:

1. Finds jobs (j) that require skills (s) the user (u) possesses.
2. OPTIONAL MATCH checks if the user knows anyone (connection) working at the company (c) hiring for that job.
3. Groups the results and ranks them first by the number of matching skills, then by the number of inside contacts the user has.

Deployment

- **Backend:** Ready to be deployed on Render (set environment variables accordingly).
- **Frontend:** Ready to be deployed on Vercel. Ensure `VITE_API_URL` is set to the backend URL.

Screenshots